Sustainability as a Software Engineering Problem

Aryan Thanki

University of Waikato Hamilton, New Zealand

at314@students.waikato.ac.nz

Abstract

The increasing digitalisation of software distribution has led to a significant rise in the number of applications published across major domains, such as the Google Play Store, Apple App Store, and GitHub. However, when applications reach the end of their maintenance lifecycle, they leave behind large volumes of abandoned software that occupy digital infrastructure with no sustained worth. This research investigates software abandonment as a sustainability issue by conducting data scraping on a bulk scale across the three aforementioned digital ecosystems. Automated Node.js-written scrapers were developed to collect metadata from the Google Play Store, Apple App Store and GitHub repositories, with filters applied to select results based on update recency and popularity to gauge the prevalence of outdated software at risk of abandonment. Results revealed trends of low engagement and infrequently updated projects, with approximately 9% of sample Play Store applications and over 50% of App Store applications meeting criteria for abandonment. These findings highlighted several issues in how digital platforms manage and control their existing applications and demonstrate the need for lifecycle-aware green software engineering strategies.

Keywords: Sustainability, Software Lifecycle, App Abandonment, Google Play Store, Apple App Store, Data Scraping, GitHub

List of Abbreviations

ICT	Information and Communication Technology
SDLC	Software Development Life Cycle
API	Application Programming Interface
OS	Operating System
SDK	Software Development Kit
IDE	Integrated Development Environment
REST	Representational State Transfer
CSV	Comma-Separated Values
JSON	JavaScript Object Notation
URL	Uniform Resource Locator
SaaS	Software-as-a-Service
kB	kilobytes

1. Introduction

Modern application stores serve as portfolios for a vast array of publicly available applications and repositories across various software platforms. While this scale allows for global creative innovation, it also carries the risk of software oversaturation and potential abandonment. Applications that no longer receive updates or user engagement but continue to exist in app stores take up unnecessary storage, energy, and maintenance resources without providing any significant value. From a sustainability perspective, these abandoned applications do not just represent wasted development effort but also a buildup of digital waste, a term used to describe unmaintained or obsolete software that continues to occupy storage, energy and data resources without benefit. This growing accumulation represents an unexplored area of sustainability within software engineering.

Historically, sustainability in Information and Communication Technology (ICT) has typically been discussed in the context of hardware efficiency, with the focus on power usage or material waste (Naumann et al., 2011). However, as software infrastructure has grown in scale and complexity, the environmental aspects of software have received attention (Becker et al., 2015; Venters et al., 2014). Frameworks like the GREENSOFT model (Naumann et al., 2011) and the Karlskrona Manifesto for Sustainability Design (Becker et al., 2015) have developed steps for extending software sustainability into the full breadth of the Software Development Life Cycle (SDLC). Despite this progress, there is still a lack of practical and measurable indicators for software sustainability.

This research addresses this topic by examining software abandonment through large-scale measurement. It focuses on data sourced from three major platforms: the Google Play Store, the Apple App Store and GitHub. Data extraction is performed by creating automated Node.js-based web scrapers to collect large quantities of application and repository metadata. This investigation applies abandonment criteria based on two specific factors. Update recency and adoption indicators, such as install count or review counts. By quantifying how many applications and repositories show signs of neglect, this study aims to reveal how inefficient current maintenance practices are and how this data-driven evidence can be used to contribute to developing sustainable software lifecycle policies.

2. Background and Policy Context

Software hosted on large distribution platforms, like the Google Play Store and the Apple App Store, usually has policies designed to determine visibility, discoverability, and long-term retention. These policies determine how digital marketplace platforms can handle outdated or non-compliant applications and decide whether an app is fit to remain publicly available for users or become effectively abandoned. Comprehending these fundamental regulations is crucial for formulating data scraping strategies and distinguishing between active and obsolete applications.

2.1 Google Play Store Policies

Google maintains strict guidelines on the applications that are allowed to be shown in the Play Store. The specific policy "*Target API level requirements for Google Play apps*" is an annually changing documentation that decides the minimum requirements for applications to remain discoverable on the Play Store (Android., 2025). Currently, as of August 2025, all new submissions must target Android 15, also referred to as API level 35, while existing apps must update to target Android level 14 to continue being freely listed. Applications that fail to meet these requirements lose the right to be discoverable to users running higher Android OS versions, thus hindering their ability to provide service without removing them from being hosted on Google's infrastructure.

This enforcement method establishes a rolling threshold, causing older applications that receive no maintenance or updates to eventually drop below the compliance level. From a sustainability viewpoint, these outdated applications still exist as objects of data, consuming storage and review resources despite not having an active growing user base. Consequently, they represent development inefficiency and digital waste, making the Play Store policy an important ingredient for evaluating abandonment.

2.2 Apple App Store Policies

Overall, Apple's ecosystem adheres to a similar lifecycle of App Store requirements, albeit with a structure that is significantly harsher in certain aspects. As of April 2025, all new iOS and iPadOS applications that are to be submitted into the App Store must be built with Xcode 16 using the iOS 18 SDK (Apple., 2025). Should developers choose a different cross-platform programming engine for development, their application must be built and compiled in the IDE Xcode version 16 for it to be submitted successfully. Developers that fail to meet this minimum requirement for their applications will find the updates for that specific app are blocked until the compliance level is met. Existing applications may remain on the app store if they are not updated for a period. Apple will

conduct periodic reviews to identify applications that are at "risk of removal" and ensure they receive updates eventually. Developers of apps that have not been updated in three years and have achieved a low download threshold over a 12-month period will receive notice of delisting. In this notice, developers are given 90 days to update their application to meet Apple's current compliance requirements. For this study, Xcode 16 and the iOS 18 SDK are the current requirements for building and compiling applications. If not, and updates are submitted within this period that meet those requirements, the app becomes unsearchable in the App Store catalogue but still accessible for existing users. This structured lifecycle ensures that inactivity is measured and creates a direct boundary between active and abandoned applications.

From a sustainability perspective, the model that Apple imposes on its developers is a far more aggressive form of digital organisation that ensures quality and security simultaneously. This method ensures that a significant number of applications that don't meet the security standards new Apple users would expect from their device will be removed because of maintenance neglect. This cycle of replacement and removal is parallel to hardware obsolescence, demonstrating a direct connection between the traditional hardware-focused waste in sustainable manufacturing.

2.3 Implications for Sustainable Software

The information provided by both the Google Play and Apple App Store policies illustrates how lifecycle management is critical towards building a secure marketplace that continuously provides applications users can feel safe with, directly influencing how sustainability is controlled. By enforcing compliance requirements that progressively increase over time, both platforms generate methods for observing how software can reach a state of abandonment. Understanding these regulations provides the necessary parameters for determining the boundaries between active and stagnated applications.

In this research, these policy guidelines form the foundation of the scraping criteria by defining when an application can be reasonably considered as outdated and at risk of abandonment. From a sustainability standpoint, such policies reflect an attempt at quality control and the persistent accumulation of outdated digital products that consume resources despite not being in use. Understanding this environment provides the foundation for analysing abandonment patterns on a large scale and assessing how sustainability can contribute to these trends.

3. Methodology

3.1 Research Design

This research employs a mixed quantitative approach focused on large-scale data scraping across three different platforms: the Google Play Store, Apple App Store and GitHub. The goal of this research is to identify measurable indicators of software abandonment by collecting publicly available metadata at bulk scale. Each scraping process will operate independently for its respective platform, using a series of Node.js-based programs, allowing for concurrent collection of unique data from their specific digital marketplace. This method was chosen based on the need to examine abandonment in a way that is quantitative, replicable, and objective.

Two key indicators were chosen for assessing software inactivity: update frequency and adoption rate. These criteria were chosen because they clearly measured developer engagement and user retention. Applications that hadn't been updated since July 2024 and showed low adoption, defined as less than 1000 installations or 20 reviews depending on the platform, were at risk of abandonment. Earlier figures from Google Play showed that 68.07% of apps never achieve 1,000 downloads (Joseph, S., 2024). This number indicates that most apps fail to garner sufficient traction for developers to consider actively maintaining them.

The choice of a scraping tool was a critical decision that required careful consideration. The google-play-scraper (Olano, F., 2015) and apple-app-scraper (Olano, F., 2016) were chosen as the main way to collect data due to their lightweight structure, ability to provide consistent results, and lack of authentication requirements. Alternative options like Puppeteer (Puppeteer., 2022) or SerpAPI (SerpAPI, 2016) were investigated but were not suitable for the bulk-scale research this investigation required. Puppeteer, a well-known web scraper, was considered, as it is powerful for browser automation, but was too inefficient in its process, as it rendered full web webpages for each query, slowing down the process for information that wouldn't be required. This, when scaled together with the bulk requirement, made it ineffective for the given task. SerpAPI was also not selected because its pay-per-request pricing model and strict rate-limiting hindered continuous scraping on a large scale, despite having access to multiple different APIs across various app stores. Similarly, the Google Play Developer Console API and the App Store Connect API, directly linked to the sources for investigation, were also unsuitable given that their access is limited to applications owned by the authenticated developer and not the broader respective application catalogue. In contrast, google-play-scraper and app-store-scraper offered stable, open-source solutions capable of handling scale and concurrency with more freedom than other tools. Unlike the mobile application stores, the selection for a GitHub scraper was more of a smooth process, as Octokit, the official GitHub REST API allowed metadata collection from the publicly available repositories without the required permissions from repository owners (GitHub., 2022).

3.2 Google Play Store

For the Google Play Store, the scraping process was implemented using the google-play-scraper Node.js module. This library allowed the extraction of structured metadata without requiring direct authentication, making it ideal for large-scale data collection. The search results from the Google Play Store are inherently biased towards applications that are recently updated or popular, a direct counter to the preferred target group of the study, so additional measures were needed to obtain results. To gather listings that contained older, less successful applications, multiple keyword seeds and category identifiers were used, allowing for a wider distribution of search results across various application types.

The scraper collected data fields, including application ID, title, developer, genre, score, ratings, installation count, update date, version number, ability to offer in-app purchases, and whether the application was free or not. To prevent throttling from numerous concurrent events, the p-limit library was connected to manage asynchronous query execution and ensure that the scraper remains compliant without inducing the rate-limit threshold. To ensure data was collected without the operation exceeding its execution time, the scraper was separated into two different programs. The initial program scraped applications that did not meet the installation count criteria. The Google Play Store separates installation count values in batches of multiples of five and two. For the context of our study, the chosen filter applications would have an installation count of either "1+", "5+", "10+", "50+", "100+", or "500+". Selecting an additional install count of "1,000+" would risk polluting the data apps that had an installation range of 1,000-4,999, a value outside the intended adoption boundary for low-engagement applications. With the first scraping complete, the output was saved into a CSV file containing the raw dataset. A secondary Node.js script developed for this study was then applied and executed to process the collected CSV data according to the second criterion of update frequency, filtering applications last updated before July 2024. Applications that had not been updated since July 2024 were retained, as these represented the category of applications least likely to be updated by developers.

The filtered dataset consisted exclusively of applications that met both the installation and update inactivity thresholds. This approach ensured that applications could be scraped reliably without experiencing a timeout. A final conversion script was used to transform the CSV file into a JSON

format, which improved data accessibility, visibility, and capability for subsequent analytical processes.

Initial testing of the google-play-scraper yielded 816 applications that met the first condition of fewer than 1000 installations. After applying the second condition of infrequent updates, the resulting dataset was reduced to 72 applications. This considerable decrease was expected because the Play Store's algorithm puts apps that are updated often and have a lot of installs at the top, making it harder to find older, less popular apps to extract. Additionally, expanding the sample size to a greater degree would result in rate limits being met, causing the scraping to fail and produce no results regardless of the time put in. However, despite this limitation, the resulting dataset captured a representative subset of the low adoption and potentially abandoned applications, aligned with the project's sustainability-focused objectives.

3.3 Apple App Store

For the Apple App Store, the scraping process was implemented using the app-store-scraper Node.js module, developed by the same developer as the google-play-scraper. This library provided the same structural simplicity, allowing for efficient retrieval of application metadata across multiple categories without needing API authentication. In contrast to the Google Play Store, the Apple App Store does not publicly display installation counts to avoid artificial download inflation, so review counts were used as a proxy metric for user adoption instead. Applications with fewer than 20 reviews and a last updated date before July 2024 were chosen as potential indicators of abandonment.

The fields of metadata extracted included application ID, title, developer name, genre, score, review count, update date, days since last update, version, and URL. Similarly to the Play Store process, the p-limit library was used to manage concurrency and ensure stable operation during the bulk scraping process. Keeping the proxy metric in mind, the program was completed in two parts, similar to the scraping process used for the Google Play Store. The first scrape extracted application metadata with fewer than 20 reviews. This data was then run through a secondary filtering script written for this study to isolate applications last updated before July 2024. Both times, the data was exported as CSV and later converted to JSON for further analysis.

The Apple App Store's stricter application policies and enforced SDK requirements resulted in fewer total results after the first extract but a higher density of outdated applications overall. Initially scraping revealed 192 applications met the low-review threshold, and after filtering for update frequency, this value yielded 98 applications in the final dataset. This higher ratio of abandoned applications compared to the Google Play Store dataset can be attributed to Apple's unbiased search engine and rigid compliance structure for outdated apps. Despite the small dataset, the results effectively reflect the broader trend of developer disengagement in low-adoption iOS applications, supporting the overall aim of this study.

3.4 GitHub Repositories

Unlike the mobile application stores, the GitHub scraping process was not designed for bulk-scale data extraction but rather for targeted analysis of large-scale software projects. The objective was to identify repositories that represented realistic production-level software that showed signs of increased inactivity, therefore aligning with the project's broader goals of evaluating abandonment on substantial open-source projects. To achieve this, the GitHub REST API was used in conjunction with GitHub's advanced search syntax to filter based on metadata like last push date, star count and archival status (GitHub., 2022). The star count metric was included as a proxy for understanding the level of community engagement and project popularity, like installation counts for applications. It represents a measurable value for how many users have shown interest in or endorsed the repository.

Lower star counts could indicate limited visibility and reduce the likelihood of continued maintenance, making it a relevant indicator for possible abandonment.

Initially, the approach taken was to utilise publicly available lists showing the top 1,000 repositories by stars, moving backwards with the assumption that repositories with fewer stars were more likely to show reduced update frequency. However, this was ineffective for identifying genuine inactive projects, as there was not a specific pattern correlation between star count and update frequency. Additionally, regardless of whether repositories had high or low star counts, many of the results were academic or documentation-based repositories rather than practical software projects. These repositories were left out because they lacked active software development or maintenance and offered no useful information for software sustainability. To increase the number of satisfactory projects appearing, the search selection process was improved to include a search query filter with a last push date before July 2024, limiting the results to projects that hadn't been updated in at least a year and were in the scope of the investigation.

In the resulting pool, repositories were manually checked to verify that they weren't archived and represented legitimate software projects, such as SaaS platforms, tools, or applications, rather than personal portfolios, book collections, or instructional material. This manual verification ensured that each repository was once actively developed for a practical purpose.

Through this process, a total of 25 repositories were selected to form the GitHub dataset. For each repository, metadata was collected with fields like name, description, stars, forks, number of open issues, watchers, size in kB, programming language, licence, last push data, date of creation, last updated, topics, and archival status using the GitHub REST API and exported directly into JSON format. The resulting dataset provided a representative view of how open-source projects can reach inactivity despite remaining publicly accessible.

4. Results

The data collected from the three platforms, Google Play Store, Apple App Store and GitHub, were processed through similar filtering steps to ensure consistent comparison. Each dataset was evaluated using two key indicators: update recency and adoption rate. Applications or repositories that had not been updated since July 2024 and exhibited low engagement were categorised as at risk of being abandoned.

4.1 Summary of Collected Data

Table 1 summarises the results for each digital platform before and after the abandonment filter was applied. It presents the initial sample size collected from each digital platform, the number of entries that met the inactivity criteria after filtering by the secondary script, the corresponding percentage of that sample, and the key indicators and metrics used for each dataset. The percentages shown in Table 1 represent the proportion of inactive applications within the scraped sample, rather than the total number of applications on each platform that meet the abandonment criteria.

Platform	Initial Sample Size	Filtered	Filtered Percentage of Initial Sample	Update Frequency	Adoption Metric
Google Play Store	816 applications	76 applications	9.3%	Last Update < July 2024	<1,000 Installations
Apple App Store	192 applications	96 applications	51.0%	Last Update < July 2024	< 20 Reviews
GitHub Repositories	25 repositories	25 repositories	100.0%	Last Push < July 2024	N/A

Table 1: Summary of scraped and filtered digital platforms dataset

These results reveal that while the total number of applications retrieved differed significantly across platforms, all three environments contained a noticeable proportion of unmaintained or low-activity software. Of the three, the Apple App Store dataset contained the highest relative inactivity rate. Apple's policies, which retain outdated apps for three years before enforcing updates, reflect these findings.

4.2 Temporal Distribution of Updates

To visualise software maintenance behaviour across time, Fig. 1 demonstrates the comparative timestamps of last updated and last pushed of all three platforms.

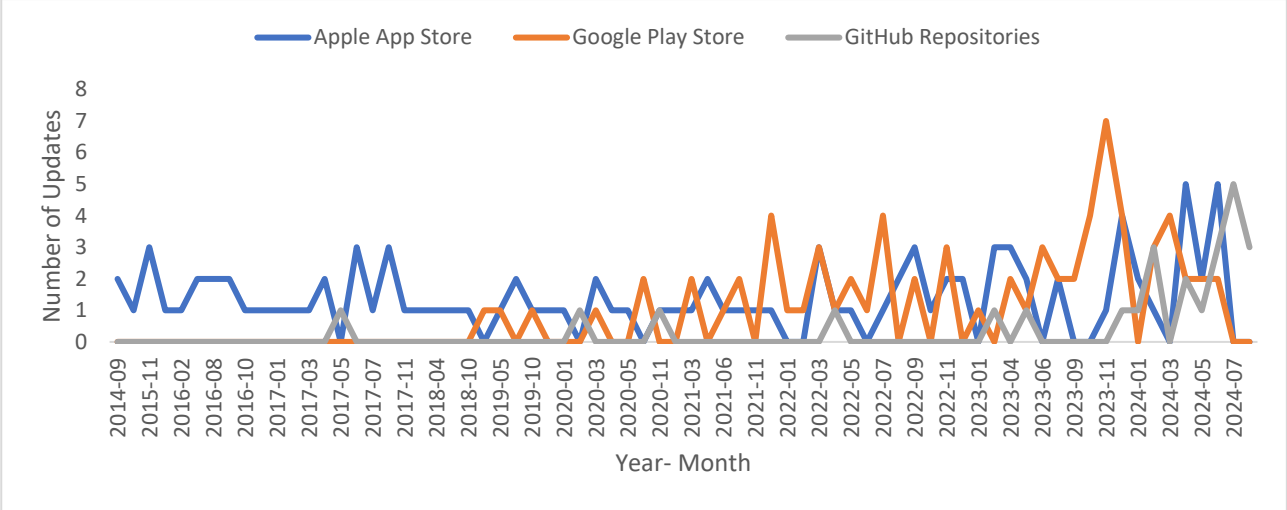


Figure 1 – Update Frequency of Scraped Data over Time

Fig. 1 illustrates the temporal distribution comparatively of last update or last push dates obtained from the scraped datasets across all three platforms. The graphs show how frequently applications on the Google Play Store and Apple App Store and repositories on GitHub were last maintained, revealing differences in maintenance activity over time.

The plot reveals three distinct patterns.

1. Google Play Store (Yellow) – Update frequency is concentrated between 2020 and 2024, displaying irregular peaks compared to Apple. The sharp increase at the end of 2023 signifies the annual policy renewal, similar to the current API level 35 requirement, which demonstrates how policy enforcement causes older applications to become dormant and inactive.
2. Apple App Store (Blue) - A steady low level of update frequency is visible from 2014 to 2020 with occasional sporadic peaks that are mostly skewed after 2022. This indicates a consistent baseline of maintenance with a steady surge before finally giving in to SDK policy constraints. An intriguing note is that applications with a last update as far back as 2014 exist yet remain on the app store, indicating that even after not updating for over three years, the application must have passed a significant domain threshold to remain existing but has less than 20 reviews.
3. GitHub Repositories (Green) – Update events remain minimal across the entire period, with only scattered activity spikes after 2022. The low overall amplitude suggests that the repositories have ended their regular maintenance and entered an inactive state.

Together, the trends indicate how policy-driven maintenance cycles in closed ecosystems like the Google Play Store and the Apple App Store demonstrate reaching a forced end-of-life state, whereas open repositories decline more naturally as time passes.

4.3 Cross Platform Comparison

The comparative research across all three digital platforms showcased a persistent pattern of inactivity, supporting the idea of extensive software abandonment. Despite the differences in how each platform is governed, each system demonstrates a pathway that causes applications or repositories to switch from active maintenance to inactivity.

In the Google Play Store, abandonment happens slowly, as low-install applications receive limited visibility in search results, reducing their discoverability and discouraging developers from continuing to invest effort and time in them. This feedback loop creates a growing list of low-adoption applications that continue to exist on the platform but remain effectively invisible for most users. In contrast, the Apple App Store demonstrates a stronger periodic lifecycle where stricter policies drive out applications that fail to meet thresholds of maintenance, becoming hidden or delisted consequently. This results in a higher proportion of outdated applications and reflects Apple's preference for quality and security over long-term software preservation. Meanwhile, GitHub repositories exemplify how publicly available open-source projects remain accessible indefinitely regardless of activity level. While this allows for potential reuse, it also results in an accumulation of inactive repositories, occupying storage and resources with no additional value. Together these differing lifecycle behaviours indicate a shared outcome: large volumes of unmaintained digital products persist beyond their functional lifespan, highlighting the need for green software engineering practices.

5. Discussion

The results from this study reveal that software abandonment is not connected solely to any specific platform or marketplace but is a recurring outcome of how current software lifecycles are managed. Each of the three examined environments, Google Play Store, Apple App Store and GitHub, shows different mechanisms that lead to similar states of inactivity. This consistent pattern across platforms highlights the direct connection between software abandonment and sustainability, while emphasising the importance of not treating it as a standalone technical issue.

From a sustainability perspective, the long-term retention of inactive applications and repositories is a growing form of digital waste that occupies copious amounts of digital infrastructure, including storage, bandwidth and indexing resources, without providing a single benefit. Although the direct environmental footprint of a single application is negligible, its cumulative effect on a multitude of applications and repositories across diverse platforms results in measurable inefficiencies in energy and data management within large-scale data centres.

The policy and governance structures of each platform also reflect differences in abandonment behaviours. Google's popularity bias model reflects a typical market dynamic, where a lack of visibility leads to the elimination of less used software without direct intervention. Apple's stricter model regularly purges applications using SDK compliance updates as the enforcement barrier. Both approaches, however, prioritise security and platform integrity over sustainability and longevity in development, leading to a continuous cycle of creation and removal instead of reuse and repurposing. In comparison, GitHub's open-source nature is a different challenge, as repositories can exist indefinitely, remaining publicly available regardless of whether they are maintained or not. While the system supports historical preservation, it enables the accumulation of unmaintained codebases with no benefit. As software improvements come into play and dependencies evolve, these repositories will lose relevance and create fragmentation in the open-source landscape. The lack of a structured

governance model for this type of platform mirrors waste management in physical systems, an accumulation of waste without processes in place for renewal or recycling.

These findings align with broader practices shown in existing frameworks such as GREENSOFT model (Naumann et al., 2011) or the Karlskrona Manifesto for Sustainability Design (Becker et al., 2015), both emphasising how sustainability should be incorporated into every stage of the SDLC. Both frameworks prioritise extending software usability and advanced maintenance over operational efficiency. Using these principles on a platform scale would require new policies to identify inactive software specifically based on key indicators and provide resources to ensure developers will invest in repurposing and maintaining older codebases.

Overall, this research determines how digital ecosystems have evolved to prioritise functionality, security, and governance over managing software until the end of its life. Addressing this gap requires shifting sustainable thinking towards being more software-centric and recognising that unmaintained applications are a real form of digital waste. This opens new opportunities for sustainable interventions, including code reuse, reducing inefficient hosting, and creating new green end-of-life standards for digital platforms.

6. Conclusion

This study examined software abandonment as a sustainability issue through large-scale data scraping of the Google Play Store, Apple App Store and GitHub. Despite differences in policy structures, all three platforms showed similar outcomes of long-term software abandonment. These findings confirm that limited maintenance and ineffective end-of-life management primarily drive software inactivity, a systemic problem that contributes to digital waste. By incorporating sustainability principles into the SDLC, such as reuse, repurposing and improved archival standards, it is possible to reduce the accumulation of unmaintained digital products and promote longer-lived software systems.

Acknowledgements

This research was completed as part of the Bachelor of Engineering (Honours) programme at the University of Waikato. A sincere thanks to Associate Professor Dr Judy Bowen for her supervision, guidance and continued feedback in the journey of this project.

References

Android. (2025, August 31). Meet Google Play's target API level requirement. Retrieved from Android Developers website: https://developer.android.com/google/play/requirements/target-sdk

Apple. (2025, April 24). SDK minimum requirements - Upcoming Requirements - Apple Developer. Retrieved from Apple.com website: https://developer.apple.com/news/upcoming-requirements/?id=02212025a

Becker, C., Chitchyan, R., Duboc, L., Easterbrook, S., Penzenstadler, B., Seyff, N., & Venters, C. C. (2015). Sustainability Design and Software: The Karlskrona Manifesto. 2015 IEEE/ACM 37th IEEE International Conference on Software Engineering. https://doi.org/10.1109/icse.2015.179

GitHub. (2022, November 28). GitHub REST API. Retrieved from GitHub Docs website: https://docs.github.com/en/rest?apiVersion=2022-11-28

Joseph, S. (2024, September 8). The difference between apps that win and fail. Medium. https://medium.com/beyond-agile-leadership/the-difference-between-apps-that-win-and-fail-b4f4bb647b61

Naumann, S., Dick, M., Kern, E., & Johann, T. (2011). The GREENSOFT Model: A reference model for green and sustainable software and its engineering. Sustainable Computing: Informatics and Systems, 1(4), 294–304. https://doi.org/10.1016/j.suscom.2011.06.004

Olano, F. (2015). GitHub - facundoolano/google-play-scraper: Node.js scraper to get data from Google Play. Retrieved from GitHub website: https://github.com/facundoolano/google-play-scraper?tab=readme-ov-file

Olano, F. (2016). GitHub - facundoolano/app-store-scraper: scrape data from the itunes app store. Retrieved from GitHub website: https://github.com/facundoolano/app-store-scraper

Puppeteer. (2022, February 4). Retrieved from GitHub website: https://github.com/puppeteer/puppeteer

SerpAPI. (2016). Retrieved October 7, 2025, from GitHub website: https://github.com/serpapi/

Venters, C. C., Lau, L., Griffiths, M. K., Holmes, V., Ward, R. R., Jay, C., Dibsdale, C. E., & Xu, J. (2014). The Blind Men and the Elephant: Towards an Empirical Evaluation Framework for Software Sustainability. Journal of Open Research Software, 2(1). https://doi.org/10.5334/jors.ao